

2. Istio 서비스 메시

1. 서비스 메시와 Istio 소개

소프트웨어 개발의 진화와 서비스 메시의 필요성을 이해한다.

1.1. 소프트웨어 개발의 변화와 마이크로서비스의 등장

1. 과거의 소프트웨어 개발 방식:

- 2000년대 이전에는 소프트웨어 개발이 매우 프로세스 중심적이고 느렸다.
- 프로젝트 하나를 완료하는 데 20년까지 걸리기도 했으며, 요구사항과 실제 제공 시점 사이에 큰 시간 격차가 있었다.
- 기업 상황 변화로 프로젝트가 중단되거나 막대한 비용 낭비가 발생하는 경우가 많았다.

2. 애자일 선언과 변화의 시작:

- 2001년, 17명의 소프트웨어 전문가들이 애자일 선언문을 발표하며 개발 방식의 근본적인 변화를 주장했다.
- 이들은 프로세스보다 개인과 상호작용, 문서보다 작동하는 소프트웨어, 계약 협상보다 고객 협업, 계획 준수보다 변화 대응을 더 중요하게 여겼다.

3. 마이크로서비스 아키텍처의 등장 배경:

- 고객 협업을 최우선으로 하고 실험 기반의 비즈니스 모델 발전이 중요해졌다.
- 하나의 큰 앱에서 문제가 발생하면 전체 시스템이 마비되는 위험을 줄이기 위해, 애플리케이션을 작게 설계하여 실험 중 위험을 격리하고 더 빠르고 빈번하게 배포하는 방식이 선호되었다.
- 기존의 통합 시스템 아키텍처는 혁신과 민첩성을 가로막는 큰 문제가 되었다.

1.2. 모놀리식 아키텍처의 한계

1. 모놀리식 아키텍처의 특징:

- 앱의 모든 기능을 동시에 배포해야 하며, 모든 개별 기능에 대해 통합된 접근 방식을 취한다.
- 모든 기능이 거의 동일한 코드베이스를 공유하고, 기능 간 명확한 경계가 없으며, 긴밀하게 결합되어 단일 프로세스처럼 작동할 수 있다.

- 일반적으로 단일 데이터베이스를 사용하여 큰 병목 현상을 야기한다.

2. 모놀리식 애플리케이션의 문제점:

- **확장의 어려움:** 특정 모듈만 확장하거나 코드 수정 없이 시스템에서 제외하는 것이 불가능하다.
- **기술 스택의 제약:** 새로운 팀이 다른 언어를 사용하려 해도 통합된 아키텍처 때문에 어려움을 겪는다.
- **기능 추가의 복잡성:** 일부 사용자 대상 테스트를 위해 새로운 기능을 시도하는 것조차 전체 애플리케이션을 두 가지 버전으로 배포해야 하는 등 복잡하다.
- **통제 불능 상태:** 느슨한 규칙으로 시작된 시스템은 시간이 지남에 따라 "진흙탕에 빠진 것처럼 엉망"이 될 수 있다.

3. 모놀리식에서 마이크로서비스로의 전환:

- 모놀리식 아키텍처를 마이크로서비스로 리팩토링하는 것은 문화적, 기술적, 조직적 노력이 필요한 어려운 작업이다.
- 각 모듈이 독립적인 애플리케이션으로 분리되어, 각기 다른 언어로 개발되고 자율성을 가질 수 있다.
- 느슨한 결합 덕분에 서비스 간 장애가 격리되고, 각 부분을 쉽게 모니터링, 변경, 롤백할 수 있어 복원력이 향상된다.

1.3. 마이크로서비스의 도전 과제와 서비스 메시의 역할

1. 마이크로서비스의 이상과 현실:

- 이상적으로 마이크로서비스는 단일 책임 원칙을 따라야 한다.
- 하지만 네트워킹, 인증, 권한 부여, 로깅, 모니터링 등 횡단 관심사(cross-cutting concerns)가 각 서비스에 코딩되면서 "비대 마이크로서비스 문제"가 발생한다.
- 이는 마이크로서비스 설계의 주된 이유인 작고 독립적인 구성 요소를 방해한다.

2. 마이크로서비스 환경의 복잡성:

- 서비스 간 통신, 보안 관리, 문제 추적 등이 복잡해진다.
- 너무 많은 작은 서비스가 분산되어 있어 문제 해결이 어려워진다.
- 전통적인 운영 방식으로는 복잡성이 증가하여 병목 현상이 될 수 있다.

3. 서비스 메시의 등장:

- 서비스 메시는 마이크로서비스 아키텍처에서 코드를 변경하지 않고도 서비스 간 통신을 처리하는 전용 인프라 계층이다.

- 각 마이크로서비스에 모든 요구 사항을 내장하는 대신, 사이드카 컨테이너 형태의 프록시로 대체한다.
- 프록시들은 데이터 플레인을 통해 서로 통신하고, 컨트롤 플레인과도 통신한다.
- 컨트롤 플레인이 모든 네트워킹 로직을 비즈니스 코드에서 추상화하여 관리한다.
- 서비스 메시는 서비스 간 통신 방식을 동적으로 구성하고, 상호 TLS를 적용하여 워크로드를 안전하게 보호하며, 애플리케이션의 엔드 투 엔드 동작 파악을 돕는다.

2. Istio 설치 및 기본 구성

Istio를 설치하고 기본 설정을 이해한다.

2.1. Istio의 아키텍처와 구성 요소

1. Istio 소개:

- 무료 오픈 소스 서비스 메시로, 서비스를 안전하게 연결하고 모니터링하는 효율적인 방법을 제공한다.
- Kubernetes 및 기존 워크로드와 호환되며, 범용 트래픽 관리, 원격 측정, 보안 기능을 제공한다.

2. 데이터 플레인:

- 각 마이크로서비스에 사이드카 컨테이너로 배포된 Envoy 프록시와 이들 간의 통신을 의미한다.
- Istio는 오픈 소스 고성능 프록시인 Envoy를 사용한다.

3. 컨트롤 플레인:

- 프록시와 통신하는 서버 측 구성 요소이다.
- 초기에는 Citadel, Pilot, Galley로 구성되었으나, 현재는 STOD라는 단일 데몬으로 통합되었다.
- Citadel은 인증서 생성을, Pilot은 서비스 검색을, Galley는 구성 파일 유효성 검사를 담당했다.
- 각 서비스 또는 Pod에는 Istio 에이전트와 Envoy 프록시가 별도로 존재하며, Istio 에이전트는 Envoy 프록시에 비밀 키를 전달한다.

2.2. Istio 설치 방법

1. 설치 도구:

- `istioctl` 명령줄 유틸리티, Istio 오퍼레이터, Helm 패키지를 사용할 수 있다.
- 이 교육에서는 `istioctl` 을 사용하여 설치한다.

2. 설치 과정:

- `istioctl install` 명령을 실행하고 설치 프로필을 지정한다.
- 데모 프로필을 사용하여 설치를 진행한다.
- 설치 시 `istio-system` 네임스페이스에 `istiod` 배포 형태로 클러스터에 배포된다.
- `istiod` 에는 Citadel, Pilot, Galley 외에도 Istio Ingress Gateway와 Istio Egress Gateway가 포함된다.

3. 설치 확인:

- 설치 완료 후 `istioctl verify-install` 명령으로 설치 상태를 확인한다.
- Kubernetes CRD(Custom Resource Definitions)도 확인할 수 있다.

2.3. 로컬 환경 설정 및 Istio 설치 (Minikube 예시)

1. Minikube 설정:

- 시스템에 Minikube가 설치되어 있어야 한다.
- `minikube start` 명령으로 클러스터를 생성한다. Docker 드라이버 사용 시 네트워크 제한으로 인한 오류가 발생할 수 있다.
- 이 경우 VM 기반 드라이버를 사용하고 Ingress 애드온을 활성화하는 것이 권장된다.

2. Istio curl 설치:

- `istioctl` 클라이언트를 다운로드하여 PATH에 추가한다.
- 다운로드한 Istio 패키지에는 샘플 애플리케이션 등이 포함되어 있다.

3. Istio 설치 및 확인:

- `istioctl install --set profile=demo` 명령으로 Istio를 클러스터에 설치한다.
- 설치 후 `istioctl verify-install` 명령으로 설치를 확인한다.
- 설치 시 클러스터 역할, 클러스터 역할 바인딩, CRD, Istio 배포 등이 생성된다.

2.4. 네임스페이스별 Istio 사이드카 주입 활성화

1. 사이드카 주입의 필요성:

- Istio 설치 후에도 Pod에 사이드카 프록시 컨테이너가 자동으로 추가되지 않는 경우가 있다.
- `istioctl analyze` 명령으로 원인을 분석하면 "Istio 주입이 활성화되어 있지 않다"는 메시지가 나타날 수 있다.

2. 네임스페이스 레이블 설정:

- Istio가 특정 네임스페이스에 배포된 애플리케이션에 사이드카를 주입하도록 하려면, 해당 네임스페이스 수준에서 Istio 사이드카 주입을 명시적으로 활성화해야 한다.
- `kubectl label namespace <namespace-name> istio-injection=enable` 명령을 사용하여 사이드카 주입을 활성화한다.
- 비활성화하려면 `istio-injection=disabled` 로 설정한다.

3. 주입 활성화 후 확인:

- 사이드카 주입을 활성화한 후 애플리케이션을 다시 배포하면, 각 Pod에 사이드카 프록시 컨테이너가 자동으로 주입된다.

3. Istio 게이트웨이와 가상 서비스 구성

외부 트래픽을 서비스 메시로 유입시키고 마이크로서비스로 라우팅하는 방법을 이해한다.

3.1. 외부 트래픽 관리: Ingress vs. Istio 게이트웨이

1. Kubernetes Ingress:

- 클러스터로 들어오는 트래픽을 제어하며, Ingress 컨트롤러와 라우팅 규칙을 통해 구현된다.
- 예를 들어, `bookingfor.app` 호스트 이름으로 들어오는 트래픽을 Product Page 서비스로 라우팅하는 규칙을 구성할 수 있다.

2. Istio 게이트웨이:

- Kubernetes Ingress보다 권장되는 방식으로, 고급 모니터링 및 라우팅 규칙을 지원한다.
- 서비스 메시의 에지에 위치하며, 서비스 메시로 들어오고 나가는 트래픽을 관리하는 주요 구성 요소이다.
- Istio 설치 시 Istio Ingress Gateway와 Istio Egress Gateway가 함께 배포된다.
- Kubernetes Ingress가 Nginx 컨트롤러를 사용하는 반면, Istio는 Envoy 프록시를 사용하여 Ingress 게이트웨이를 배포한다.

- Ingress 및 Egress 게이트웨이는 독립형 Envoy 프록시이며, 서비스 메시의 에지에서 작동한다.

3. 목표:

- Istio Ingress Gateway 컨트롤러를 통해 들어오는 모든 트래픽을 포착하여, 특정 호스트 이름(`bookingfor.app`)의 트래픽을 Product Page 서비스로 라우팅하는 것을 목표로 한다.

3.2. 게이트웨이(Gateway) 객체 생성

1. 게이트웨이 객체 정의:

- `gateway` 객체를 생성하여 특정 호스트 이름과 포트로 들어오는 HTTP 트래픽을 처리하도록 설정한다.
- 예시: `booking` 이라는 이름의 게이트웨이는 `bookinfo.app` 호스트 이름의 80번 포트로 들어오는 HTTP 트래픽을 처리한다.

2. 셀렉터 설정:

- 어떤 Ingress Gateway 컨트롤러를 사용할지 지정하기 위해 셀렉터를 설정한다.
- 기본 Istio Ingress Gateway 컨트롤러를 사용하려면 셀렉터를 `ingressgateway` 로 설정한다.

3. 게이트웨이 적용 및 확인:

- `kubectl apply -f <gateway-file.yaml>` 명령으로 게이트웨이 객체를 적용한다.
- `kubectl get gateway` 명령으로 생성된 게이트웨이 목록을 확인하고, `kubectl describe gateway` 명령으로 상세 정보를 볼 수 있다.
- 게이트웨이가 생성되면, 해당 앱의 URL로 들어오는 트래픽을 캡처할 준비가 된다.

3.3. 가상 서비스(VirtualService)를 통한 라우팅 규칙 정의

1. 가상 서비스의 역할:

- 게이트웨이를 통해 서비스 메시로 들어오는 트래픽에 대한 라우팅 규칙 집합을 정의한다.
- 풍부한 트래픽 라우팅 옵션을 제공하며, 서비스의 여러 버전에 걸쳐 트래픽을 관리할 수 있다.
- 표준 경로와 정규 표현식 경로를 모두 지원한다.

2. 가상 서비스 생성:

- `virtualservice` 종류로 객체를 생성하며, `networking.istio.io/v1alpha3` API 버전을 사용한다.
- `metadata.name` 에 서비스 이름을 지정한다 (예: `bookinfo`).
- `spec.hosts` 에 트래픽을 처리할 호스트 이름을 지정한다 (예: `bookinfo.app`).
- `spec.gateways` 에 연결할 게이트웨이 이름을 지정한다 (예: `bookinfo-gateway`).

3. 라우팅 규칙 정의:

- `spec.http` 섹션에서 라우팅 규칙을 정의한다.
- `match` 섹션에서 일치해야 하는 URI를 지정한다 (`exact` 또는 `prefix`).
- `route` 섹션에서 일치하는 트래픽을 라우팅할 대상을 지정한다.
- 예시: `bookingfor.app` 호스트로 들어오는 `/static` , `/api` , `/v1/products` 등의 URI 패턴과 일치하는 트래픽을 Product Page 서비스로 라우팅한다.

4. 가상 서비스와 게이트웨이의 연동:

- 가상 서비스는 게이트웨이와 바인딩되어야 하며, 가상 서비스의 호스트와 게이트웨이의 호스트는 일치해야 한다.
- 가상 서비스가 생성되면, 컨트롤 플레인은 이 구성을 모든 사이드카 프록시에 적용한다.

4. 서비스 버전 관리 및 트래픽 분배

다양한 서비스 버전으로 트래픽을 유연하게 분배하고 관리하는 방법을 이해한다.

4.1. Kubernetes에서의 점진적 배포와 한계

1. 점진적 배포 시나리오:

- 새로운 버전의 리뷰 서비스(v2, v3)를 개발하고, 점진적으로 트래픽을 분배하여 테스트한다.
- 처음에는 트래픽의 일부만 새 버전에 보내고, 테스트 후 전체 트래픽을 전환한다.

2. Kubernetes 서비스의 트래픽 분산:

- Kubernetes 서비스는 여러 Pod에 트래픽을 균등하게 분산한다 (예: 75% v1, 25% v2).
- 분산 비율을 변경하려면 각 서비스에서 사용 가능한 Pod 수를 조정해야 한다.

- 이 방식은 세밀한 비율 제어(예: 99% vs 1%)가 어렵고, Pod 수에 따라 분산 비율이 제한된다.

3. Kubernetes 방식의 한계:

- Pod 수가 적을 경우 원하는 비율로 트래픽을 분산하기 어렵다.
- 세밀한 트래픽 제어가 필요한 경우(예: 1%만 새 버전으로 보내기) 한계가 있다.

4.2. 가상 서비스를 이용한 트래픽 가중치 분배

1. 가상 서비스로 트래픽 분배:

- 서비스 대신 가상 서비스를 생성하여 트래픽 분배 서브셋(subset)에 대한 목적지 경로를 정의하고 가중치를 설정한다.
- 예시: 리뷰 서비스에 대해 v1에 99%, v2에 1%의 가중치를 설정하여 트래픽을 라우팅한다.
- 인스턴스 수와 상관없이 가상 서비스 구성을 통해 트래픽 분배를 제어할 수 있다.

2. 서브셋(Subset)의 개념:

- 레이블을 사용하여 여러 객체를 그룹화하는 방법으로, 서비스의 별도 버전을 정의한다.

3. 트래픽 가중치 변경 데모:

- 리뷰 서비스의 버전별 가중치를 변경하고, 쓰기 규칙을 만들어 특정 사용자에게 다른 버전을 보여준다.
- `reviews` 가상 서비스에서 `http.route` 섹션에 두 개의 대상 경로(v1, v2)를 정의하고 가중치를 75:25로 설정한다.
- Kiali 인터페이스에서 가중치 분배를 확인하고, 트래픽을 생성하여 변경 사항을 검증한다.

4.3. 사용자 기반 라우팅 (A/B 테스트 및 카나리 릴리스)

1. 특정 사용자에게 특정 버전 노출:

- 프로젝트 소유자가 일반 사용자에게는 기본 버전을, 특정 사용자 그룹(예: CodeCloud 사용자)에게는 특별 버전을 제공하고 싶을 때 유용하다.

2. `match` 섹션을 이용한 규칙 정의:

- HTTP 섹션 아래에 `match` 섹션을 추가하여 요청 헤더를 기반으로 라우팅 규칙을 설정한다.

- 예시: 요청 헤더에 `end user` 가 `code-cloud` 와 일치하면 리뷰 버전 2로, 그렇지 않으면 리뷰 버전 1로 라우팅한다.

3. 요청 헤더 생성:

- 애플리케이션의 로그인 기능을 사용하여 요청에 `end user` 헤더를 추가할 수 있다.
- 로그인하면 해당 사용자는 설정된 특별 버전(리뷰 v2)을 보게 되고, 로그아웃하면 기본 버전(리뷰 v1)으로 돌아간다.

4. 테스트 사용자 그룹 설정:

- 새로운 매칭 섹션을 만들어 테스트 사용자에게 `test-user` 라는 이름을 부여하고, 이들에게 최신 버전(v3)을 보여주도록 설정할 수 있다.
- 이를 통해 특정 사용자 그룹을 대상으로 새로운 버전을 테스트하고 피드백을 받을 수 있다.

4.4. 대상 규칙(DestinationRule)을 이용한 서브셋 정의 및 로드 밸런싱

1. 대상 규칙의 역할:

- 트래픽이 특정 서비스로 라우팅된 후, 라우팅 정책을 적용하는 역할을 한다.
- 가상 서비스에서 정의된 서브셋(subset)을 실제로 정의하고 구성한다.

2. 서브셋 정의:

- `destinationrule` 종류로 객체를 생성하고, `host` 에 서비스 이름을 지정한다.
- `subsets` 섹션에서 각 버전별 서브셋을 정의하고, 해당 Pod에 설정된 레이블을 지정한다 (예: `version: v1`, `version: v2`).

3. 로드 밸런싱 정책:

- 기본적으로 Istio는 라운드 로빈 방식으로 트래픽을 분산한다.
- `trafficPolicy` 를 `load` 로 지정하여 사용자 정의할 수 있다.
- 단순 패스스루, 랜덤 등 다양한 로드 밸런싱 정책을 설정할 수 있다.
- 특정 서브셋에 대해 별도의 로드 밸런싱 정책을 재정의할 수 있다.

4. 주의사항:

- `host` 필드에 짧은 서비스 이름을 사용할 경우, Istio는 규칙의 네임스페이스를 기반으로 해석하므로 네임스페이스를 고려해야 한다.

- 구성 오류를 방지하기 위해 짧은 이름 대신 정규화된 도메인 이름을 사용하는 것이 좋다.

5. 오류 주입 및 재시도 메커니즘

애플리케이션의 복원력을 테스트하고 오류를 처리하는 방법을 이해한다.

5.1. 오류 주입(Fault Injection)

1. 오류 주입의 목적:

- 테스트 접근 방식으로, 정책이 효율적으로 실행되고 지나치게 제한적이지 않은지 확인하는 데 도움이 된다.
- 애플리케이션의 오류 처리 메커니즘이 예상대로 작동하는지 확인하기 위해 오류를 주입한다.

2. 오류 유형:

- **지연(Delay):** 요청 처리를 의도적으로 지연시킨다.
 - 예시: 평점 가상 서비스에 요청의 10%에 대해 5초의 지연을 추가한다.
- **중단(Abort):** 요청을 거부하고 특정 오류 코드를 반환하여 오류를 시뮬레이션한다.

5.2. 타임아웃(Timeout) 설정

1. 타임아웃의 필요성:

- 마이크로서비스 아키텍처에서는 서비스 속도 저하나 오류 발생을 예상해야 한다.
- 한 서비스의 속도 저하는 다른 서비스에 영향을 미쳐 연쇄적인 파급 효과를 일으킬 수 있다.
- 서비스 응답 시간이 너무 오래 걸릴 경우, 종속 서비스가 영원히 기다리지 않도록 일정 시간이 지나면 오류를 반환해야 한다.

2. 타임아웃 설정:

- 가상 서비스 수준에서 타임아웃 옵션을 추가하고 원하는 시간을 설정한다.
- 예시: Product Page 서비스에 타임아웃을 3초로 설정하여 요청 처리 시간이 3초를 초과하면 요청을 거부하도록 한다.

3. 타임아웃 테스트:

- 오류 주입을 사용하여 상세 서비스에 고정 지연 시간을 추가하고, 트래픽의 50%에 대해 이 오류를 주입하여 타임아웃 설정을 테스트한다.

5.3. 재시도(Retry) 메커니즘

1. 재시도 기능:

- 서비스가 다른 서비스 접근에 실패했을 때, 애플리케이션 코드 내에서 재처리를 구현할 필요 없이 가상 서비스가 해당 작업을 다시 시도하도록 구성할 수 있다.

2. 재시도 설정:

- `retry` 설정을 통해 시도 횟수와 시도 간격을 지정할 수 있다.
- Istio는 기본적으로 25밀리초 간격으로 재시도 동작을 설정한다.
- 오류가 발생하기 전에 두 번 재시도하도록 설정할 수 있다.
- `perTryTimeout` 옵션을 사용하여 각 시도 사이의 지연 시간도 설정할 수 있다.

3. 재시도 설정 위치:

- 타임아웃과 마찬가지로 재시도 설정은 가상 서비스 수준에서 지정할 수 있다.